

# Design Document

## Vision-Based Desktop Automation with Dynamic Icon Grounding

Notepad automation on Windows 10/11 (1920×1080) · ScreenSeeker grounding (arXiv:2504.07981)

### 1. Objective & Assumptions

The system launches Notepad by *seeing* its desktop icon rather than by a fixed path, then writes the first ten JSONPlaceholder posts to disk. The icon may sit anywhere on the desktop, at any Windows icon size, under a light or dark theme. The core requirement is **general, scalable visual grounding** — locating any described on-screen element from pixels alone — so that the same approach works for any icon or button, including unexpected pop-ups never seen in advance.

- **Environment:** Windows 10/11, single primary display. DPI scaling is handled by making the process per-monitor DPI-aware so screenshot pixels and cursor coordinates share one space.
- **Prerequisites:** a Notepad shortcut on the Desktop and a Desktop/tjm-project output folder.
- **Grounding model:** a vision-language model (VLM). The reference implementation uses one Gemini model for both roles; the paper-faithful backend (OS-Atlas grounder + GPT-4o planner) is a drop-in alternative.
- **Non-goals / excluded shortcuts:** template matching, OCR-label matching, and accessibility-tree lookups are deliberately avoided — each needs the target's image, text, or a selector in advance and cannot generalise to the size/theme/pop-up variation being evaluated.

### 2. System Design

The system separates three concerns: **perception** (a VLM that turns a screenshot plus a natural-language description into coordinates), **deterministic control** (keyboard/clipboard automation and the fixed per-post workflow), and **robustness** (confidence thresholds, verified launches, pop-up handling, and a circuit breaker). Grounding — the graded capability — is used only where a click on a described element is unavoidable; everything achievable by keyboard bypasses it for speed and reliability.

| Module            | Responsibility                                                                                         |
|-------------------|--------------------------------------------------------------------------------------------------------|
| search.py         | ScreenSeeker engine: fast single-shot path + recursive best-first search with three-way verification.  |
| scoring.py        | Candidate scoring: box dilation, Eq.1 Gaussian centrality ( $\sigma=0.3$ ), NMS, ranking. Unit-tested. |
| transforms.py     | Per-crop coordinate remapping back to original pixels (the detail the paper omits).                    |
| gemini_backend.py | One VLM implementing both Grounder and Planner (ground / position-inference / verify / classify).      |
| automation.py     | Deterministic screenshot, click, type, save, close; DPI awareness; focused-window guard.               |
| popups.py         | Hybrid pop-up handler: keyword/OCR detection, keystroke dismissal, VLM fallback, per-post recovery.    |
| main.py           | Orchestrator: per-post loop, launch verification, debug capture, circuit breaker, run report.          |

### 3. Grounding Strategy (ScreenSeeker)

Grounding models are accurate on small crops but weak on full high-resolution screens, where the target is a tiny fraction of the image. The strategy therefore **reduces the search space before grounding**, following ScreenSpot-Pro's ScreenSeeker:

- **Fast path (common case):** ground the full screenshot once; if the planner verifies the box is the target, click immediately — one to two model calls for a clear desktop icon.
- **Recursive fallback:** a planner proposes candidate regions; the grounder votes; candidates are dilated, scored by centrality, de-duplicated with NMS, and explored best-first, depth-first. At each leaf the planner verifies with a three-way check.
- **Three-way verdict drives backtracking:** `is_target` ends the search; `target_elsewhere` keeps searching sibling regions (the target is nearby); `target_not_found` unwinds to the parent (wrong region).
- **Stop conditions:** maximum depth and minimum crop size bound the cost; exhausting all candidates yields an explicit failure rather than a blind click.
- **Gating by verification, not confidence:** a general VLM's self-reported confidence is unreliable (often zero even for a correct box), so acceptance is gated by the planner's `verify()` call, not by a confidence threshold. The search also never dead-ends — if the planner proposes no regions, the current crop is grounded directly.

Coordinate remapping is treated as a first-class concern: every crop has its own pixel frame, so an affine (offset + scale) transform maps each prediction back to original-screen coordinates before clicking. This is unit-tested with round-trip checks.

### 4. Automation Workflow

Each of the ten posts runs the same self-contained cycle, with grounding used only for the icon click:

- Reveal the desktop (minimise all windows), capture a fresh screenshot, and ground the Notepad icon.
- Double-click, then poll until Notepad is the focused window, retrying the click once — the launch is verified before anything is typed.
- Paste the post as `"Title: {title}\n\n{body}"` (clipboard paste, not per-keystroke, for speed and exactness).
- Save via `Ctrl+S` → type the full `tjm-project/post_{id}.txt` path → `Enter`, clearing any dialog before and after.
- Close with `Alt+F4` — but only when Notepad is confirmed in front, then loop with a fresh screenshot.

### 5. Error Handling & Robustness

- **Detection failure:** if the icon is never grounded, the system takes no action on screen — it saves a debug screenshot, logs it, records the skip, recovers to a clean desktop, and continues. It never clicks a guessed coordinate.
- **Launch guard:** typing and `Alt+F4` only proceed once the focused window is Notepad, so a missed click can never type into the void or send `Alt+F4` to the bare desktop (which would open the Shut Down dialog).
- **Known pop-ups:** the small, finite set of Notepad dialogs (overwrite, unsaved-changes, path-error) is detected by OCR keyword matching — deliberately chosen over a VLM call because OCR is far faster — and dismissed with a single `Enter` (the default button: Yes / Save / OK), instantly and deterministically.
- **Unknown pop-ups:** an unrecognised dialog is screenshotted, passed to the VLM to choose the safest non-destructive dismiss button (Cancel/No/Close/OK, never Delete/Shut Down), clicked, verified, and logged; `Esc` is the fallback and a safe per-post skip is the last resort.

- **Circuit breaker:** a single failure skips one post so the batch survives; three consecutive failures abort the run, since that signals a systemic problem.
- **Run report:** every run writes a timestamped report file (saved / skipped-with-reasons / aborted, plus any unknown pop-ups and their debug-screenshot paths) and prints the same summary, so nothing fails silently.

## 6. Tradeoffs

| Decision                                    | Rationale / tradeoff                                                                                                                                      |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| VLM grounding vs. template/OCR/UIA          | Grounding generalises to position, size, theme, and unseen pop-ups; template/OCR/selectors are faster but brittle and are the shortcuts the task forbids. |
| Fast path then recursion                    | Common case is 1-2 calls; the bounded recursion (depth 2, top-2) protects the hard case without ballooning latency.                                       |
| Keyboard for known dialogs, VLM for unknown | Known dialogs are dismissed instantly by keystroke; only novel pop-ups pay the cost of a model call — speed where possible, generality where needed.      |
| Grounding only for clicks                   | Typing/saving/closing use global shortcuts, minimising dependence on pixel-perfect vision and improving reliability.                                      |
| One Gemini model vs. paper backend          | A single free VLM (no GPU) serves both roles; the OS-Atlas + GPT-4o backend offers higher grounding precision at hardware cost. Both are pluggable.       |

## 7. Evaluation & Bonus Coverage

Success is measured as in the paper: a prediction is correct if its centre point falls inside the target's ground-truth box ("centre-in-box" — it measures whether the click would land). A small hand-labelled set of desktop screenshots is scored, sliced by icon position, size, and theme, and compared across the bare grounder, ReGround, and the full engine to confirm the search helps.

- **Multiple icons:** a rich target description plus candidate ranking and the verify step disambiguate near-duplicates (e.g. Notepad vs. Notepad++).
- **Icon sizes:** cropping enlarges a small icon's relative size, directly addressing the paper's small-target failure mode.
- **Light/dark themes:** the target is described by form rather than colour, so grounding is theme-invariant where template/OCR methods break.

In summary, grounding is model-driven perception, flow control is deterministic scripting, and the system re-screenshots before every action — turning unexpected pop-ups and missed clicks into recoverable events rather than failures.

## Appendix — Workflow & Failure-Handling Flow

The diagram below traces the per-post flow end to end. Blue boxes are deterministic steps, the orange box is the VLM grounding call, diamonds are decisions, and every red path is a failure/exception route. Known pop-ups are resolved by fast OCR detection + a keystroke; unknown pop-ups escalate to the VLM; and any unrecoverable failure saves a debug screenshot, logs, skips the post, and feeds the three-strikes circuit breaker.

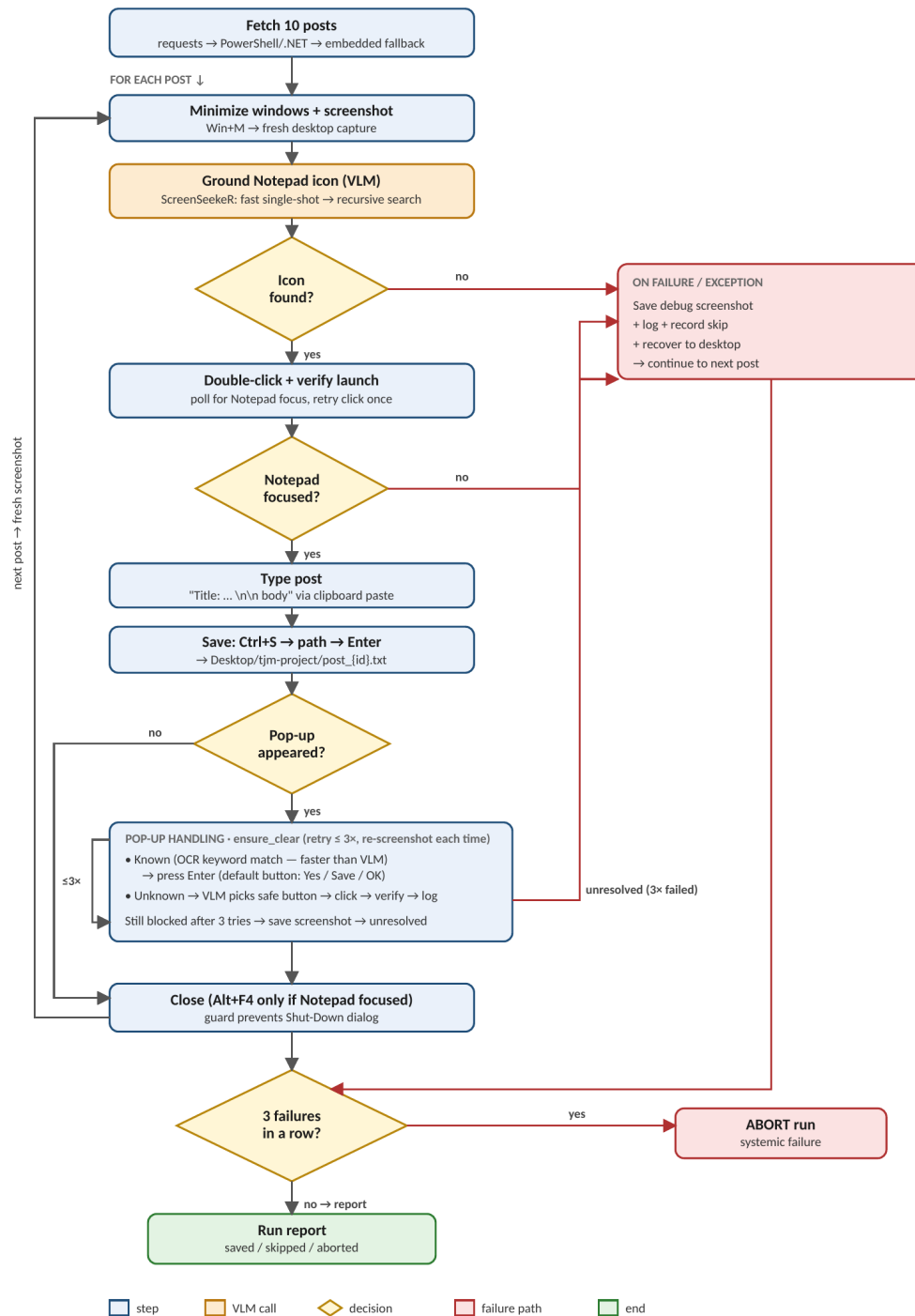


Figure 1. Per-post automation flow with grounding, pop-up handling, and failure branches.